

Corollary 7.6 (Soundness and completeness): *A proposition α is provable if and only if it is valid, i.e., $\vdash_H \alpha \Leftrightarrow \models \alpha$.*

Remarks 7.7:

(i) *On modus ponens:* If α has a tableau proof and $\alpha \rightarrow \beta$ has a tableau proof, then α and $\alpha \rightarrow \beta$ are both valid by the soundness theorem. As modus ponens preserves validity, β is also valid. Thus, by the completeness theorem for tableau proofs, β has a tableau proof. There is actually an algorithm for getting a tableau proof for β from such proofs for α and $\alpha \rightarrow \beta$. This is known as the Gentzen Hauptsatz (principle theorem) and is too long to prove here. Modus ponens is also called the cut rule and this theorem is therefore referred to as a cut elimination theorem.

(ii) *On theorems:* A *theorem* is any proposition that is provable. So any proposition that occurs as an element in a proof is a theorem. We usually think of the *conclusion* as being the last element of a proof but, any initial segment of a proof is also a proof.

(iii) *Choice of axioms:* The corollary says that the axioms are complete in the sense that we can prove any valid proposition from them by repeated applications of modus ponens. On the other hand, since the axioms are valid and modus ponens preserves validity, every theorem (i.e., every proposition provable in this system) has a tableau proof. Thus, tableau proofs are sufficient and so are the axioms and rules of inference listed above. One could have more axioms (or fewer) or more (or other) rules of inference or both. Sometimes it is a matter of taste, other times a matter of expediency (e.g., what makes various proofs easier). The key point is that whatever the proof system, there is really only one set of theorems, the valid propositions.

(iv) *Efficiency:* Proving theorems efficiently from such a system of axioms and rules may be somewhat tricky since you often have to guess which axiom to use rather than having a systematic procedure as is the case for the tableaux. The source of this problem is having a plethora of axioms from which to choose. The Hilbert-style proof system presented here has many axioms and few rules. Other systems that reverse the emphasis are Gentzen systems and natural deduction systems. These are much more relevant to automatic theorem proving and, in their intuitionistic or constructivist forms, to producing systems that have the property of always being able to produce a proof or counterexample for any given proposition (as discussed at the end of §5).

8 Resolution

The proof method underlying PROLOG and most automatic theorem provers is a particularly simple and efficient system of axioms and rules called *resolution*. Like the system presented in §7, resolution has only one rule. It reduces the large amount of the guesswork involved in producing a proof by essentially eliminating all axioms. (Actually it incorporates them automatically via various formatting rules but as far as the work of producing the proof is concerned, this almost

amounts to their elimination.) The resolution method, like our version of the tableau method, is a refutation procedure. That is, it tries to show that the given formula is unsatisfiable. It begins by assuming that the formula of interest is in conjunctive normal form (see Exercises 3.3 and 4.10-11). In typical computer science treatments this form is called *clausal form* and the associated terminology is as follows:

Definition 8.1:

- (i) A *literal* ℓ is a propositional letter p or its negation $\neg p$. If ℓ is p or $\neg p$, we write $\bar{\ell}$ for $\neg p$ or p , respectively. The propositional letters are also called *positive literals* and their negations *negative literals*.
- (ii) A *clause* C is a finite set of literals (which you should think of as the disjunction of its elements). As we think of C as being true iff one of its elements is true, the *empty clause* $\square$ is always false — it has no true element.
- (iii) A *formula* S is a (not necessarily finite) set of clauses (which you should think of as the conjunction of its elements). As we think of a formula S as being true if every one of its elements is true, the *empty formula* $\emptyset$ is always true — it has no false element.
- (iv) An *assignment* $\mathcal{A}$ is a consistent set of literals, i.e., one not containing both p and $\neg p$ for any propositional letter p . (This, of course, is just the (partial) truth assignment in which those $p \in \mathcal{A}$ are assigned T and those q with $\bar{q} \in \mathcal{A}$ are assigned F .) A *complete assignment* is one containing p or $\neg p$ for every propositional letter p . It corresponds to what we called a truth assignment in Definition 3.1.
- (v) $\mathcal{A}$ *satisfies* S , $\mathcal{A} \models S$, iff $\forall C \in S (C \cap \mathcal{A} \neq \emptyset)$, i.e., the valuation induced by $\mathcal{A}$ makes every clause in S true.
- (vi) A formula S is *(un)satisfiable* if there is an (no) assignment $\mathcal{A}$ that satisfies it.

Examples 8.2:

- (i) $p, q, r, \neg p, \bar{q}(=\neg q), \bar{r}$ and $\neg\bar{q}(=q)$ are literals.
- (ii) $\{p, r\}, \{\neg q\}$ and $\{q, \neg r\}$ are clauses.
- (iii) $S = \{\{p, r\}, \{q, \neg r\}, \{\neg q\}, \{\neg p, t\}, \{s, \neg t\}\}$ is a formula that, in our original notation system, would be written as $((p \vee r) \wedge (q \vee \neg r) \wedge (\neg q) \wedge (\neg p \vee t) \wedge (s \vee \neg t))$.
- (iv) If $\mathcal{A}$ is given by $\{p, q, r, s, t\}$, i.e., the (partial) assignment such that $\mathcal{A}(p) = T = \mathcal{A}(q) = \mathcal{A}(r) = \mathcal{A}(s) = \mathcal{A}(t)$, then $\mathcal{A}$ is an assignment not satisfying the formula S in (iii). S is, however, satisfiable.

* PROLOG Notation:

Another way of thinking of clausal or conjunctive normal form is in terms of implications. Suppose we have a clause C whose *positive literals* (the propositional letters contained in C) are $A_1, \dots, A_m$ and whose *negative literals* (the propositional letters p such that $\bar{p}$ (i.e., $(\neg p)$) is an element of C) are $B_1, \dots, B_n$. The clause C is then equivalent to $A_1 \vee A_2 \vee \dots \vee A_m \vee \neg B_1 \vee \dots \vee \neg B_n$. This in turn is equivalent to $B_1 \wedge B_2 \dots \wedge B_n \rightarrow A_1 \vee \dots \vee A_m$. If there is at most one positive literal (i.e., at most one A_i) in C , then C is called a *Horn clause* (or a *program clause* if it has exactly one positive literal). If the Horn clause contains some negative literals it is a *rule*, otherwise a *fact*. A *goal clause* is one with no positive literals. It is the logic and proof theory of these clauses (which we analyze in Section 10) that is the heart of PROLOG. (PROLOG is, however, not limited to propositional letters. It also allows for variables in its literals. We elaborate on this when we deal with predicate logic in Chapter II.)

The standard notations in PROLOG reverse the order used in $\rightarrow$ and instead use either $\leftarrow$ or $:-$ which are read "if". Occurrences of the $\wedge$ symbol are replaced by commas. Thus, $A_1 :- B_1, B_2, \dots, B_n$ or $A_1 \leftarrow B_1, \dots, B_n$ is read (and means) A_1 if (B_1 and B_2 and ... and B_n). In terms of generating deductions or writing programs one thinks of the assertion of a clause C such as $A_1 :- B_1, \dots, B_n$, as specifying conditions under which A_1 is true. We are usually interested in establishing some result. Thus, A_1 is called the *goal* of the clause C (or at times the *head* of C) and $B_1, \dots, B_n$ the subgoals (or *body* and with this terminology the symbol " $:-$ " is called the *neck*) of C . The idea is that C tells us that to establish A we should first establish each of $B_1, \dots, B_n$. Along with the goal — subgoal terminology, come the terms succeed and fail. One says a goal A succeeds if it is true, or more precisely from the programming point of view, if we have a proof of A . Otherwise we say the goal fails. Be warned, however, that this terminology of success and failure is (at least for now) somewhat imprecise.

It is worth noting what these views imply for the meaning of the degenerate cases of the notation $:-$, i.e., when $n = 0$ or $m = 0$. If $m = 0$, then $:- B_1, \dots, B_n$ (or $\leftarrow B_1, \dots, B_n$), called a *goal clause*, is equivalent to $\neg B_1 \vee \dots \vee \neg B_n$, i.e., it asserts that one of the B_i fails (is false). If $n = 0$, then $A_1 :-$ (or $A_1 \leftarrow$), called a *unit clause*, is equivalent to simply A_1 ; thus this notation simply says that A_1 succeeds (is true).

The *resolution rule* is much like a version of modus ponens called cut. Modus ponens (see §7) says that from α and $\alpha \rightarrow \beta$ one can infer β . In this format, the cut rule says that from $\alpha \vee \gamma$ and $\neg\alpha \vee \beta$ infer $\gamma \vee \beta$. Thus, cut is somewhat more general than modus ponens in that it allows one to carry along the extra proposition γ . Resolution is a restricted version of cut in which α must be a literal while β and γ must be clauses.

Definition 8.3 (Resolution): In our current terminology, we say that, from clauses C_1 and C_2 of the form $\{\ell\} \sqcup C'_1$ and $\{\bar{\ell}\} \sqcup C'_2$, infer $C = C'_1 \cup C'_2$ which is called a *resolvent* of C_1 and C_2 . (Here ℓ is any literal and $\sqcup$ means that we are taking a union of disjoint sets.) We may also call C_1 and C_2 the *parent* and C their *child* and say that we *resolved on* (the literal) ℓ .

(Note that, compared to the classical form of the cut rule, the resolution rule also eliminates redundancies, i.e., letters common to C_1 and C_2 . This takes the place of certain axioms in a classical proof system such as the Hilbert-style one of §7.)

Resolution is, of course, a sound rule, that is, it preserves satisfiability for every truth assignment. If some assignment satisfies both C_1 and C_2 , whatever it does for ℓ , the literal on which one resolves, it must satisfy one of C'_1 or C'_2 and hence the resolvent $C'_1 \cup C'_2$. (This argument is formalized in Lemma 8.12.) The resolution rule can thus be used as the basis of a sound proof procedure.

Definition 8.4: A (resolution) deduction or proof of C from a given formula S is a finite sequence $C_1, C_2, \dots, C_n = C$ of clauses such that each C_i is either a member of S or a resolvent of clauses C_j, C_k for $j, k < i$. If there is such a deduction, we say that C is (resolution) provable from S and write $S \vdash_{\mathcal{R}} C$. A deduction of $\square$ from S is called a (resolution) refutation of S . If there is such a deduction we say that S is (resolution) refutable and write $S \vdash_{\mathcal{R}} \square$.

Warning: A resolution refutation of S gives a proof of $\square$ from S . As $\square$ is always false, we should think of this as showing that S can never be true, i.e., S is unsatisfiable. This is the soundness theorem (Theorem 8.11).

Examples 8.5:

- (i) From $\{p, r\}$ and $\{\neg q, \neg r\}$ conclude $\{p, \neg q\}$ by resolution (on r).
- (ii) From $\{p, q, \neg r, s\}$ and $\{\neg p, q, r, t\}$ we could conclude either $\{q, \neg r, s, r, t\}$ or $\{p, q, s, \neg p, t\}$ by resolution (on p or r), respectively. Of course, both of these clauses are valid and are equivalent to the empty formula.

A more useful picture of a resolution proof is as a tree of deductions rather than just the sequence described above.

Definition 8.6: A resolution tree proof of C from S is a labeled binary tree T with the following properties:

- (i) The root of T is labeled C .
- (ii) The leaves of T are labeled with elements of S .
- (iii) If any nonleaf node σ is labeled with C_2 and its immediate successors σ_0, σ_1 are labeled with C_0, C_1 , respectively, then C_2 is a resolvent of C_0 and C_1 .

Example 8.7: Figure 16 gives a resolution tree refutation of the formula $S = \{\{p, r\}, \{q, \neg r\}, \{\neg q\}, \{\neg p, t\}, \{\neg s\}, \{s, \neg t\}\}$, i.e., a resolution tree proof of $\square$ from S :

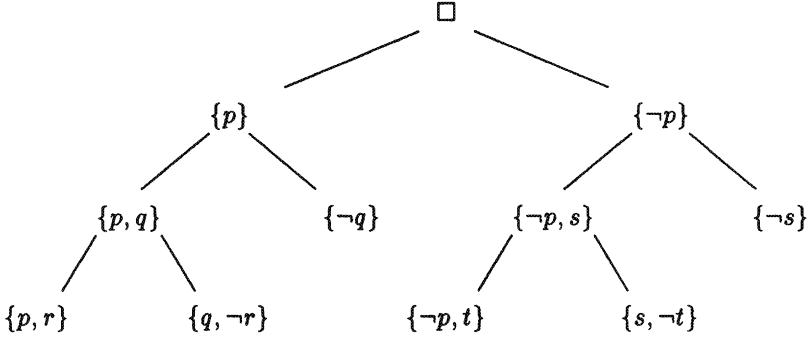


FIGURE 16.

Lemma 8.8: *C has a resolution tree proof from S if and only if there is a resolution deduction of C from S .*

Proof: $(\Rightarrow)$ List all the labels of the nodes σ of the tree proof of C from S in any order that reverses the $<$ ordering of the tree (so leaves are listed first and the root last). This sequence can be seen to be a resolution deduction of C from S by simply checking the definitions.

$(\Leftarrow)$ We proceed by induction on the length of the resolution deduction of C from S . Suppose we can get tree proofs for any deduction of length $< n$ and $C_1, \dots, C_n$ is one of length n from S . If $C_n \in S$, there is nothing to prove. If not, then C_n is the resolvent of C_i and C_j for some i and j less than n . By induction, we have tree proofs T_i and T_j of C_i and C_j . Let T_n be the tree whose root is labeled C and to whose immediate successors we attach T_i and T_j . Again, by definition, this is the desired tree proof. $\square$

Yet another picture of resolution deduction corresponds to the inductive definition of the set of theorems or clauses provable from S .

Definition 8.9: $\mathcal{R}(S)$ is the *closure of S under resolution*, i.e., the set determined by the following inductive definition:

1. If $C \in S$, $C \in \mathcal{R}(S)$.
2. If $C_1, C_2 \in \mathcal{R}(S)$ and C is a resolvent of C_1 and C_2 , then $C \in \mathcal{R}(S)$.

Proposition 8.10: *For any clause C and formula S , there is a resolution deduction of C from S iff $C \in \mathcal{R}(S)$. In particular, there is a resolution refutation of S iff $\square \in \mathcal{R}(S)$.*

Proof: Exercise 1. $\square$

The first observation to be made is that no matter how the resolution method is described, it gives a sound proof procedure.

Theorem 8.11 (Soundness of resolution): *If there is a resolution refutation of S , then S is unsatisfiable.*

We first prove a lemma which is needed for the inductive step in the proof of the theorem.

Lemma 8.12: *If the formula (i.e., set of clauses) $S = \{C_1, C_2\}$ is satisfiable and C is a resolvent of C_1 and C_2 , then C is satisfiable. Indeed, any assignment $\mathcal{A}$ satisfying S satisfies C .*

Proof: As C is a resolvent of C_1 and C_2 , there are ℓ, C'_1 and C'_2 such that $C_1 = \{\ell\} \sqcup C'_1$, $C_2 = \{\bar{\ell}\} \sqcup C'_2$ and $C = C'_1 \cup C'_2$. As $\mathcal{A}$ satisfies $\{C_1, C_2\}$, it satisfies (that is, it contains an element of) each of C_1 and C_2 . As $\mathcal{A}$ is an assignment, it cannot be the case that both $\ell \in \mathcal{A}$ and $\bar{\ell} \in \mathcal{A}$. Say $\bar{\ell} \notin \mathcal{A}$. As $\mathcal{A} \models C_2$ and $\bar{\ell} \notin \mathcal{A}$, $\mathcal{A} \models C'_2$ and so $\mathcal{A} \models C$. The proof for $\ell \notin \mathcal{A}$ just replaces C_2 by C_1 and $\bar{\ell}$ by ℓ . $\square$

Proof (of Theorem 8.11): If $C_1, \dots, C_n$ is a resolution deduction from S , then the lemma shows by induction (on n) that any assignment satisfying S satisfies every C_i . If the deduction is in fact a refutation of S , then $C_n = \square$. As no assignment can satisfy $\square$, S is unsatisfiable. $\square$

Remark 8.13: The soundness theorem and its proof could just as well have been phrased directly in terms of Definitions 8.6 or 8.9. We leave these formulations as Exercises 2 and 3.

Our next major goal is to prove that the resolution method is complete, i.e., if S is unsatisfiable, then there is a resolution refutation of S . We then want to consider ways of implementing a search for a refutation of S . We first consider using the resolution method as originally presented. We then introduce more and more restrictive versions of resolution which are designed to make the search more efficient without rendering the method either unsound or incomplete. Following this line of development, we first present a simple direct proof of the completeness of the general form of resolution given in Definition 8.3. This proof, however, relies on the (semantic form of the) compactness theorem. We then introduce and analyze a somewhat abstract description of unsatisfiability. It supplies us with a proof of the completeness theorem for resolution deduction that does not rely on the compactness theorem and a new proof of the compactness theorem. That proof of completeness is the paradigm for the completeness proofs of the restricted version of resolution presented in §9.

We begin our first path to completeness with a lemma that allows us to eliminate literals in clauses which are resolution deducible from an unsatisfiable formula S . Repeated applications of the lemma show that $\square$, the clause with no literals, is deducible from S .

Lemma 8.14: *For any formula T and any literal ℓ , let $T(\ell) = \{C \in \mathcal{R}(T) \mid \ell, \bar{\ell} \notin C\}$. If T is unsatisfiable, then so is $T(\ell)$.*

Proof: Assume T is unsatisfiable and suppose, for the sake of a contradiction, that $\mathcal{A}$ is any assignment that satisfies $T(\ell)$ and is defined on all the literals (of T) other than ℓ . Let $\mathcal{A}_1 = \mathcal{A} \cup \{\ell\}$ and $\mathcal{A}_2 = \mathcal{A} \cup \{\bar{\ell}\}$. As T is unsatisfiable, there are clauses C_1 and C_2 in T such that $\mathcal{A}_1 \not\models C_1$ and $\mathcal{A}_2 \not\models C_2$. Now as $\ell \in \mathcal{A}_1$ and $\mathcal{A}_1 \not\models C_1$, $\ell \notin C_1$. If $\bar{\ell}$ is also not in C_1 , then $C_1 \in T(\ell)$ by definition. As this would contradict our assumption that $\mathcal{A} \models T(\ell)$, $\bar{\ell} \in C_1$. Similarly, $\ell \in C_2$. Thus, we may resolve C_1 and C_2 on ℓ to get a clause D not containing ℓ and hence in $T(\ell)$. (As a resolvent of two clauses in T , D is certainly in $\mathcal{R}(T)$). Then, by our choice of $\mathcal{A}$, $\mathcal{A} \models D$. If $\mathcal{A}$ satisfies the resolvent D , however, it must satisfy one of the parents C_1 or C_2 . Thus, we have the desired contradiction. $\square$

Theorem 8.15 (Completeness of resolution): *If S is unsatisfiable, then there is a resolution refutation of S .*

Proof: By the compactness theorem (Theorem 6.13), there is a finite subset S' of S that is unsatisfiable. As any refutation deduction from S' is one from S , we may assume that S is finite, i.e., it contains only finitely many clauses. If there are only finitely many clauses in S and each clause is finite, there are only finitely many literals, say $\ell_1, \ell_2, \dots, \ell_n$ which are in any clause in S . For the rest of the proof we consider only clauses and formulas based on these n literals.

We wish to consider the set of clauses $C \in \mathcal{R}(S)$ and prove that it contains $\square$. We proceed by eliminating each literal in turn by applying Lemma 8.14. We begin with $S_n = S(\ell_n) = \{C \in \mathcal{R}(S) \mid \ell_n, \bar{\ell}_n \notin C\}$. By definition, it is a collection of resolution consequences of S none of which contain ℓ_n or $\bar{\ell}_n$. By Lemma 8.14 it is unsatisfiable. Next we let $S_{n-1} = S_n(\ell_{n-1})$. It is an unsatisfiable collection of resolution consequences of S_n (and hence of S) none of which contain ℓ_{n-1} , $\bar{\ell}_{n-1}$, ℓ_n or $\bar{\ell}_n$. Continuing in this way we define $S_{n-2}, \dots, S_0$. By repeated applications of the definitions and Lemma 8.14, we see that S_0 is an unsatisfiable set of resolution consequences of S containing no literals at all. As the only formulas with no literals are $\emptyset$ and $\{\square\}$ and $\emptyset$ is satisfiable, $\square \in S_0$. Thus, $\square$ is a resolution consequence of S as required. $\square$

We now turn to a more abstract formulation of the notions and lemmas inherent in the proof of the completeness of resolution deduction. They are needed to deal with the refinements of resolution in §9 and §10.

Definition 8.16: If S is a formula and ℓ a literal, we let

$$S^\ell = \{C - \{\bar{\ell}\} \mid C \in S \wedge \ell \notin C\}.$$

So S^ℓ consists of those clauses C of S containing neither ℓ nor $\bar{\ell}$, plus those clauses (not containing ℓ) such that $C \cup \{\bar{\ell}\} \in S$. Note that if the singleton clause $\{\bar{\ell}\}$ is in S , then $\square$ is in S^ℓ .

Admittedly, this definition seems somewhat obscure at first reading. It is based on the idea that we can analyze (the satisfiability of) S by cases. S^ℓ corresponds to the result of the analysis under the assumption that ℓ is true. $S^{\bar{\ell}}$ gives the result when ℓ is assumed false. Consider, for example, the formula S under the assumption that ℓ is true. The first point here is that, if ℓ is true, then any clause containing ℓ is satisfied since a clause is equivalent to the disjunction of its literals. As the formula S is equivalent to the conjunction of its clauses, any clause known to be true can be eliminated from S without changing its satisfiability. Thus, assuming ℓ to be true, we may omit any clause containing ℓ from S as far as satisfiability is concerned. This is precisely the point of the part of the definition of S^ℓ that restricts the clauses under consideration to those C such that $\ell \notin C$. The next point of the analysis is that, still assuming ℓ to be true, $\bar{\ell}$ can be omitted from any clause C containing it without changing the satisfiability of C . (Again C is equivalent to the disjunction of its members. If one of them is known to be false it cannot affect the satisfiability of the disjunction.) Of course, if the satisfiability of C is not affected, neither is that of the formula S containing it. This is then the point of that part of the definition of S^ℓ which says replace C by the smaller clause $C - \{\bar{\ell}\}$.

If ℓ is false, then $\bar{\ell}$ is true and the same analysis applies to $S^{\bar{\ell}}$. As one of ℓ and $\bar{\ell}$ must be true, we can argue (as we do in Lemma 8.19) that S is satisfiable if and only if one of S^ℓ and $S^{\bar{\ell}}$ is satisfiable. Thus, we can reduce the satisfiability problem for S to two similar problems for formulas S^ℓ and $S^{\bar{\ell}}$ with one less propositional letter. We can then continue this procedure by considering each of the two new formulas S^ℓ and $S^{\bar{\ell}}$. In this way, we could produce a binary tree of formulas in which we would successively eliminate one literal at each level of the tree. Every path through this tree corresponds to an assignment. The branch through S^ℓ is the one that makes ℓ true. The one through $\bar{\ell}$ is the one that makes ℓ false. If every path through the tree ends with a formula containing the empty clause $\square$, we can conclude that the original formula S was unsatisfiable. On the other hand, if not all paths lead to $\square$, then, if we successively eliminate all the literals appearing in S , either there is an infinite path along which we have eliminated every literal or at least one path ends with the empty formula $\emptyset$. In either case S is satisfiable. Indeed, the appropriate path (infinite or leading to $\emptyset$) directly supplies an assignment satisfying S .

Seen in this way, the plan of the analysis is similar to that of tableau proofs beginning with $F\alpha$ for some proposition α . There too, we attempted to analyze all ways of making α false, i.e., of verifying $F\alpha$. If they all lead to contradictions ($\otimes$), we conclude that $F\alpha$ is unsatisfiable and α is valid. Here, if all paths lead to a formula containing the unsatisfiable clause $\square$, we conclude that the formula S is unsatisfiable. On the other hand, if the tableau analysis was finished and produced a noncontradictory path, we could use that path (Lemma 5.4) to define a valuation satisfying α . In the analysis here, when we eliminate all the literals (corresponding to finishing the tableau) and are left with an infinite path or one ending with the empty formula $\emptyset$, this path itself directly supplies the assignment satisfying S .

We illustrate the construction of S^ℓ from S and the general form of this analysis by considering two examples.

Example 8.17: Let $S = \{\{p\}, \{\neg q\}, \{\neg p, \neg q\}\}$. The analysis in which we eliminate first p and then q can be represented by the tree in Figure 17:

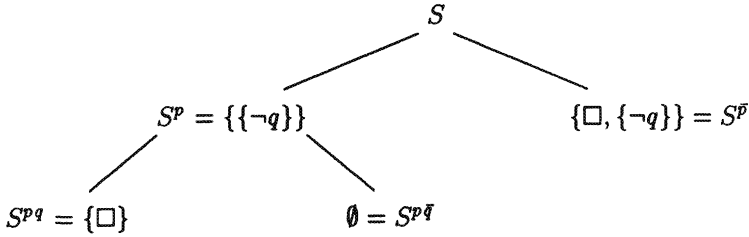


FIGURE 17.

Assuming p is true, we eliminate the clause $\{p\}$ from S and the literal $\neg p$ from the clause $\{\neg p, \neg q\}$ to get S^p on the left side of the first level of the tree. Assuming that p is false, the right side ($S^{\bar{p}}$) reduces to $\{\square, \{\neg q\}\}$ since S asserts that p is true by having $\{p\}$ as one of its clauses. At the next level, we consider q . On the left, when q is assumed true, we again get $\square$ as S^p asserts that $\neg q$ is true. On the right, where we assume that q is false, we eliminate all clauses containing $\neg q$ to get the empty formula. Thus, we have a path ending in $\emptyset$. It supplies the assignment satisfying S : Make p true and q false.

Example 8.18: Consider the formula proven unsatisfiable in Example 8.7, $S = \{\{p, r\}, \{q, \neg r\}, \{\neg q\}, \{\neg p, t\}, \{\neg s\}, \{s, \neg t\}\}$. We begin the analysis by eliminating p . When we assume p to be true, we eliminate the clauses containing p (as they are true) and omit $\neg p$ from the others (since being false $\neg p$ cannot help to satisfy them) to get $S^p = \{\{q, \neg r\}, \{\neg q\}, \{t\}, \{\neg s\}, \{s, \neg t\}\}$. On the other hand, when we assume that p is false we eliminate clauses containing $\neg p$ and remove p from the others to get $S^{\bar{p}} = \{\{r\}, \{q, \neg r\}, \{\neg q\}, \{\neg s\}, \{s, \neg t\}\}$. Figure 18 shows part of the full tree analysis.

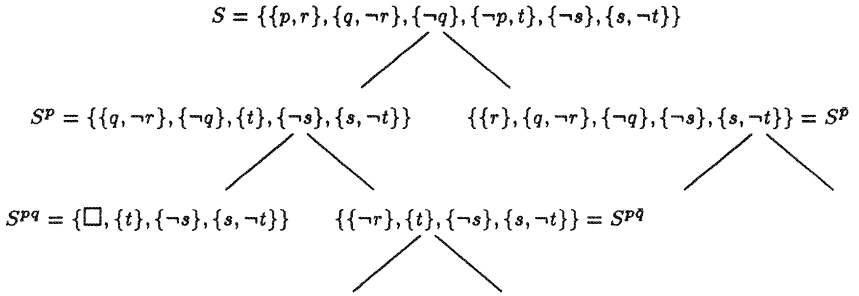


FIGURE 18.

The path of the analysis through SP^q terminates at this point since it contains $\square$ and so is unsatisfiable. The other paths displayed, however, continue. If continued, every path would eventually terminate with an unsatisfiable formula containing $\square$ as a clause. This is the analog of the proof that $S \vdash_{\mathcal{R}} \square$. We leave the completion of this analysis as Exercise 4.

We now formulate and prove the results that say that the analysis discussed above correctly captures the notion of satisfiability.

Lemma 8.19: *S is satisfiable if and only if either S^ℓ or $S^{\bar{\ell}}$ is satisfiable. (Warning: In the “if” direction the assignments are not necessarily the same.)*

Proof: ($\Rightarrow$) Suppose that $\mathcal{A} \models S$. If $\mathcal{A}$ were a complete assignment, we could conclude that it must make one of ℓ , $\bar{\ell}$ true, say ℓ . We could then show that $\mathcal{A} \models S^\ell$. If we do not wish to make this assumption on $\mathcal{A}$, we instead start with the fact that, by definition, one of ℓ or $\bar{\ell}$ does not belong to $\mathcal{A}$. For the sake of definiteness assume that $\bar{\ell} \notin \mathcal{A}$. We now also claim that $\mathcal{A} \models S^\ell$. We must show that $\mathcal{A}$ satisfies every clause in S^ℓ . Consider any $C \in S^\ell$. By the definition of S^ℓ , either $C \cup \{\bar{\ell}\} \in S$ or $C \in S$ (depending on whether or not $\bar{\ell}$ is in the clause of S which “puts” C into S^ℓ). Thus, by hypothesis, $\mathcal{A} \models C$ or $\mathcal{A} \models C \cup \{\bar{\ell}\}$. As an assignment satisfies a clause only if it contains one of its literals, there is a literal k such that either $k \in C \cap \mathcal{A}$ or $k \in (C \cup \{\bar{\ell}\}) \cap \mathcal{A}$. As $\bar{\ell} \notin \mathcal{A}$ by our assumption, in either case we must have $k \in C \cap \mathcal{A}$, i.e., $\mathcal{A} \models C$ as required. The case that $\ell \notin \mathcal{A}$ is handled similarly.

($\Leftarrow$) Suppose for definiteness that $\mathcal{A} \models S^\ell$. Now neither ℓ nor $\bar{\ell}$ appear in any clause of S^ℓ and so we may adjust $\mathcal{A}$ on ℓ as we choose without disturbing the satisfiability of S^ℓ . More precisely, if we let $\mathcal{A}' = (\mathcal{A} - \{\bar{\ell}\}) \cup \{\ell\}$, then $\mathcal{A}' \models S^\ell$ as well. We claim that $\mathcal{A}' \models S$. Consider any $C \in S$. If $\ell \in C$, then $\mathcal{A}' \models C$ as $\ell \in \mathcal{A}'$. If $\ell \notin C$ then $C - \{\bar{\ell}\} \in S^\ell$ by definition of S^ℓ . As $\mathcal{A} \models S^\ell$, there is some literal $k \in (C - \{\bar{\ell}\}) \cap \mathcal{A}$. Now $\mathcal{A}$ and $\mathcal{A}'$ differ at most at ℓ and $\bar{\ell}$. As $k \neq \ell$ or $\bar{\ell}$, we see that $k \in \mathcal{A}' \cap C$ as required. $\square$

Corollary 8.20: *S is unsatisfiable iff both S^ℓ and $S^{\bar{\ell}}$ are.*

This corollary, together with the unsatisfiability of $\square$, actually characterizes the property of unsatisfiability.

Theorem 8.21: *If $\text{UNSAT} = \{S \mid S \text{ is an unsatisfiable formula}\}$, then UNSAT is the collection $\mathcal{U}$ of formulas defined inductively by the following clauses:*

$$(i) \quad \square \in S \Rightarrow S \in \mathcal{U}$$

and

$$(ii) \quad S^\ell \in \mathcal{U} \wedge S^{\bar{\ell}} \in \mathcal{U} \Rightarrow S \in \mathcal{U}.$$

Proof: As $\square$ is unsatisfiable, UNSAT satisfies (i). By Corollary 8.20 it also satisfies (ii). Thus, $\mathcal{U} \subseteq \text{UNSAT}$. We must show that $\text{UNSAT} \subseteq \mathcal{U}$. We prove the contrapositive by showing that if $S \notin \mathcal{U}$, then S is satisfiable. Let $\{p_i\}$ list the propositional letters such that p_i or $\bar{p}_i$ occurs in a clause of S . Define by induction the sequence $\{\ell_i\}$ such that $\ell_i = p_i$ or $\bar{p}_i$ and $S^{\ell_1, \dots, \ell_i} \notin \mathcal{U}$. (Property (ii) guarantees that we can always find such an ℓ_i .) Now let $\mathcal{A} = \{\ell_i \mid i \in \mathcal{N}\}$. We claim that $\mathcal{A}$ satisfies S . Suppose $C \in S$. We must show that $C \cap \mathcal{A} \neq \emptyset$. As C is finite, there is an n such that for all propositional letters p_i occurring in C , $i < n$. If $C \cap \mathcal{A} = \emptyset$, then $\forall i < n$ ($\ell_i \notin C$) and so a clause corresponding to C is passed on to each $S^{\ell_1, \dots, \ell_i}$ for $i < n$. At each such transfer, say to $S^{\ell_1, \dots, \ell_i}$, we remove $\bar{\ell}_i$ from the clause. As all literals in C are among the $\bar{\ell}_i$, the clause deriving from C becomes $\square$ in $S^{\ell_1, \dots, \ell_n}$. By our choice of the ℓ_i , $S^{\ell_1, \dots, \ell_n} \notin \mathcal{U}$. On the other hand, any S containing $\square$ is in $\mathcal{U}$ by Clause (i) and we have our desired contradiction. $\square$

This result is the analog of Lemma 5.4. The choice of the sequence ℓ_i corresponds to the definition of the assignment in Lemma 5.4 from the signed propositional letters appearing on the noncontradictory path on the finished tableau. As there, we are building an assignment that satisfies every entry on the path being constructed. Since we eventually reach the unsatisfiable clause $\square$ in this construction, we have the desired contradiction. As for tableau proofs, this characterization of unsatisfiability is really the heart of the completeness proof of the resolution method.

Theorem 8.22 (Completeness of the resolution method): *If S is unsatisfiable, then there is a resolution refutation of S (equivalently, $\square \in \mathcal{R}(S)$).*

Proof: We proceed by induction according to the characterization of UNSAT provided by Theorem 8.21. Of course, if $\square \in S$, then $\square \in \mathcal{R}(S)$. For the inductive step, suppose that, for some ℓ and S , $\square \in \mathcal{R}(S^\ell)$ and $\square \in \mathcal{R}(S^{\bar{\ell}})$. We must show that $\square \in \mathcal{R}(S)$. By assumption, we have tree proofs T_0 and T_1 of $\square$ from S^ℓ and $S^{\bar{\ell}}$. Consider T_0 . If every leaf in T_0 is labeled with a clause in S , then T_0 is already a proof of $\square$ from S . If not, we define a tree T'_0 by changing every label C on T_0

that is above a leaf labeled with a clause not in S to $C \cup \{\bar{\ell}\}$. We claim that T'_0 is a tree proof of $\{\bar{\ell}\}$ from S . Clearly, by the definition of S^ℓ , every leaf of T'_0 is in S . We must now check that every nonleaf node of T'_0 is labeled with a resolvent C' of its immediate successors C'_0 and C'_1 . Suppose they correspond to clauses C , C_0 and C_1 , respectively, on T_0 . As T_0 is a resolution tree proof, C is a resolvent of C_0 and C_1 . Note first that no resolution in T_0 is on ℓ or $\bar{\ell}$ as neither appear in any label on T_0 (by the definition of S^ℓ). Next, consider the possible forms of clauses C'_0 , C'_1 and C' on T'_0 . If, for example, both C_0 and C_1 (and hence certainly C) are above leaves labeled with clauses not in S , then $C' = C \cup \{\bar{\ell}\}$ is the resolvent of $C'_0 = C_0 \cup \{\bar{\ell}\}$ and $C'_1 = C_1 \cup \{\bar{\ell}\}$, as is required for T'_0 to be a resolution tree proof. The other cases to consider either keep all three clauses the same in T'_0 as they were in T_0 or change C and precisely one of C_0 and C_1 by adding on $\{\bar{\ell}\}$. In all these cases C' is still clearly the resolvent of C'_0 and C'_1 and we again verify that T'_0 is a tree proof. Similarly, if we replace every label C on a node of T_1 above a leaf labeled with a clause not in S by $C \cup \{\ell\}$, we get T'_1 , a tree proof of $\{\ell\}$ from S (or, if all leaves were in S , one of $\square$). We can now define a tree proof T of $\square$ from S by simply attaching T'_0 and T'_1 to the immediate successors of the root node of T which we label with $\square$. As $\square$ is a resolvent of $\{\ell\}$ and $\{\bar{\ell}\}$, the resulting tree T is a proof of $\square$ from S . $\square$

*Compactness revisited

Of course there is no need to reprove the compactness theorem as it can be phrased solely in semantic terms. Nonetheless we offer another proof based on the characterization of UNSAT given by Theorem 8.21. It is the construction of the infinite sequence ℓ_i of literals in the proof of this theorem that corresponds to the path through the tree of assignments (given by König's lemma) in our original proof of compactness in Theorem 6.13.

Theorem 8.23 (Compactness): *If S is unsatisfiable, so is some finite subset of S .*

Proof: Let $\mathcal{T} = \{S \mid \exists S_1 \subseteq S [S_1 \text{ is finite} \wedge S_1 \text{ is unsatisfiable}]\}$. If we can show that $\mathcal{T}$ satisfies (i) and (ii) of Theorem 8.21, then we are done for it will then contain all unsatisfiable formulas.

(i) If $\square \in S$, then $S_1 = \{\square\} \subseteq S$ shows that $S \in \mathcal{T}$ as required.

(ii) Suppose $S^\ell, S^{\bar{\ell}} \in \mathcal{T}$. We must show that $S \in \mathcal{T}$. By definition of $\mathcal{T}$, S^ℓ and $S^{\bar{\ell}}$, there are finite formulas $S_1, S_2 \subseteq S$ such that $S_1^\ell \subseteq S^\ell$, $S_2^{\bar{\ell}} \subseteq S^{\bar{\ell}}$ and S_1^ℓ and $S_2^{\bar{\ell}}$ are unsatisfiable. Let $S_3 = S_1 \cup S_2$. S_3 is a finite subset of S . It suffices to show that it is unsatisfiable. If not, then there would be an assignment $\mathcal{A}$ satisfying S_3 . Now $\mathcal{A}$ must omit either ℓ or $\bar{\ell}$. Thus, $\mathcal{A}$ would satisfy either S_1^ℓ or $S_2^{\bar{\ell}}$, respectively. As it would then satisfy S_2^ℓ or $S_1^{\bar{\ell}}$ (as $S_3 \supset S_2, S_1$), we have the desired contradiction. $\square$

Exercises

1. Prove Proposition 8.10 by induction. (Hint: For one direction proceed by induction on the number of lines in the proof. For the other direction proceed by induction on the definition of $\mathcal{R}(S)$.)
2. Rephrase the proof of Theorem 8.11 (soundness) in terms of resolution tree proofs and an induction on their definition.
3. Do the same for the version of resolution deductions defined in terms of $\square \in \mathcal{R}(S)$.
4. Continue the analysis in Example 8.18 until every path terminates with a formula equivalent to the unsatisfiable clause $\square$.
5. Rewrite the following in both conjunctive normal and clausal form.
 - (a) $((A \vee B) \rightarrow (C \vee D))$
 - (b) $\neg(A \wedge B \wedge \neg C)$
 - (c) $\neg((A \wedge B) \vee (B \vee C) \vee (A \wedge C))$.
6. Which of the following clause sets are satisfiable? Give assignments satisfying them if they are. If they are not, explain why not.
 - (a) $\{\{A, B\}, \{\neg A, \neg B\}, \{\neg A, B\}\}$
 - (b) $\{\{\neg A\}, \{A, \neg B\}, \{B\}\}$
 - (c) $\{\{A\}, \square\}$
 - (d) $\{\square\}$.
7. Find all resolvents for the following pairs:
 - (a) $\{A, B\}, \{\neg A, \neg B\}$
 - (b) $\{A, \neg B\}, \{B, C, D\}$.
8. Find $\mathcal{R}(S)$ for the following sets S :
 - (a) $\{\{A, \neg B\}, \{A, B\}, \{\neg A\}\}$
 - (b) $\{\{A\}, \{B\}, \{A, B\}\}$.
9. Find a deduction of the empty clause from
 - (a) $\{\{A, \neg B, C\}, \{B, C\}, \{\neg A, C\}, \{B, \neg C\}, \{\neg B\}\}$.
10. Use resolution to show that each of the following is not satisfiable by any assignment.